applied sciences

MDPI

Article

Attention-Based Light Weight Deep Learning Models for Early Potato Disease Detection

Singara Singh Kasana *,† and Ajayraj Singh Rathore †

Department of Computer Science and Information Technology, Central University of Haryana, Mahendergarh 123031, Haryana, India; iamajayrajrathore@gmail.com
* Correspondence: singarasingh@cuh.ac.in
† These authors contributed equally to this work.

Abstract: Potato crop has become integral part of our diet due to its wide use in variety of dishes, making it an important food crop. Its importance also stems from the fact that it is one of the cheapest vegetables available throughout the year. This makes it crucial to keep potato prices affordable for developing countries where the majority of the population falls under the middle-income bracket. Consequently, there is a need to develop a robust, effective, and portable technique to detect diseases in potato plant leaves. In this work, an attention-based disease detection technique is proposed. This technique selectively focuses on specific parts of an image which reveal the disease. This technique leverages transfer learning combined with two attention modules: the channel attention module and spatial attention module. By focusing on specific parts of the images, the proposed technique is able to achieve almost similar accuracy with significantly fewer parameters. The proposed technique has been validated using four pre-trained models: DenseNet169, XceptionNet, MobileNet, and VGG16. All of these models are able to achieve almost the same level of training and validation accuracy, around 90–97%, even after reducing the number of parameters by 40–50%. It shows that the proposed technique effectively reduces model complexity without compromising performance.

Keywords: attention; DenseNet169; XceptionNet; MobileNet; VGG16; precision; recall; F_1 score

check for updates

Citation: Kasana, S.S.; Rathore, A.S. Attention-Based Light Weight Deep Learning Models for Early Potato Disease Detection. *Appl. Sci.* **2024**, *14*, 8038. https://doi.org/10.3390/app14178038

Academic Editors: Yu-Xiang Zhao, Chien-Hsing Chou and Yi-Zeng Hsieh

Received: 5 June 2024
Revised: 3 September 2024
Accepted: 4 September 2024
Published: 8 September 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

According to the United Nations, the projected population in the year 2050 will be around 10 billion [1]. To sustain this population, global food production needs to increase by 60%. Since potatoes are one of the most widely consumed food products, it is crucial to increase their production. To achieve this, we either need to grow more potato crops or increase the efficiency of existing farmland. The latter option is considered the better choice as it saves space and reduces the effort required to maintain a larger area. More cultivated land would mean higher fertilizer requirements, more labour, and more machinery. In countries like India, where majority of the population is involved in agriculture and related activities, it is vital for farmers to enhance their production capacity. Many farmers have limited land, and some work on other's land on a contractual basis. After rice and wheat, potatoes are the third most widely grown cash crop in India.

Potato plants can be infected by various diseases such as Early Blight, Late Blight, leaf roll virus, scab, hollow heart, etc. Among these, Late Blight and Early Blight are the most common diseases affecting potato plants. Scientifically, these diseases are known as Phytophthora Infestans and Alternaria Solani, respectively. They have drastically impacted crop production over the years, and the quality of remaining crops continues to be affected today. Timely identification of the plant disease can prevent their spread by using suitable treatments, thereby avoiding financial losses. In the past, human inspection was the sole method for detecting plant diseases. However, this approach has several drawbacks: only experts are able to identify the diseases, and human evaluation is prone to error, and the area that humans can cover is limited, etc.

Technological solutions offer a more robust and effective approach to these problems. Many researchers have already worked on the early detection of plant diseases. As technology advances, even more convenient and simpler solutions can be developed. With the advancement of technologies like Artificial Intelligence (AI) and Computer Vision (CV), disease identification is no longer limited to human inspection alone. Software applications can now instantly identify various disease. Recent developments in agricultural technology have shifted away from the conventional image processing techniques which rely on feature extraction and segmentation for disease detection to more advanced methods of deep learning. In the realm of AI, a new field known as "deep learning" has emerged which has shown superior performance, especially compared to the traditional image processing methods. It utilizes kernels, also known as filters, to extract relevant features from images. These extracted features are then used to differentiate among various plant diseases. Each plant disease exhibits a unique pattern on its leaves, distinguishing it from other types of diseases. The combination of traditional image processing methods and deep learning has proven to be an effective approach for identifying and classifying plant diseases.

1.1. Literature Review

Initially, researchers and scientists worked on multiple crops using a dataset known as PlantVillage developed by the researchers from the USA and Switzerland [2]. This region-based dataset was used to study crop disease, which can vary due to differences in leaf shape, variety of leaves, and environmental factors [3]. Atiya Khan et al. [4] proposed the use of hyperspectral imaging and advanced machine learning algorithms in agriculture. Dubey et al. [5] proposed a *K*-means clustering technique for multiclass *SVM* classification to segregating contaminated leaf sections. Lie et al. [6] combined the *SVM* and *K*-means clustering with a backpropogation neural network to identify disease in plant leaves. Many imaging-based research projects have been developed with advancements in technologies such as machine learning and deep learning [7,8]. A deep CNN model was proposed by Geetharamani and Pandian [9] to differentiate between healthy and unhealthy leaves of multiple crops. Their model was trained on the PlantVillage dataset and included 38 kinds of crops. Khamparia et al. [10] combined autoencoders and a CNN to propose a model for detecting crop diseases from multiple crops, specifically tailored to a region by training on the PlantVillage dataset. Liang et al. [11] proposed a model using ResNet50 architecture for disease detection and severity detection. This model was also trained on the PlantVillage dataset and could detect disease in multiple plants.

Many researchers have specifically focused on the potato crop using the PlantVillage dataset. Khalifa proposed a CNN-based model to detect Early Blight (EB), Late Blight (LB), and healthy images [12]. Rozaqi and Sunyoto [13] also proposed a CNN model for detecting *EB*, *LB*, and healthy images using the PlantVillage data. Iqbal et al. [14] utilized 450 images from the PlantVillage dataset to train multiple machine learning models and observed that Random Forest outperformed all other models. Singh et al. [15] used 300 images from the PlantVillage dataset, 100 images each from EB, LB, and healthy classes. They employed the GLCM feature extraction technique to extract the features from the images and implemented an SVM classifier to achieve an overall accuracy of 96%. Chakraborty et al. [16] used pre-trained models like ResNet50, MobileNet, VGG16, and VGG19 for performance evaluation on the potato leaf images of the PlantVillage dataset. The VGG19 architecture initially achieved the highest accuracy of 92.69%. After being fine-tuned, the model's accuracy improved to 97.89%. Mahum et al. [17] used a pre-trained DenseNet architecture with additional layers to classify potato blight using images from the PlantVillage dataset. The enhanced model achieved an accuracy of 97.2%, outperforming the basic DenseNet architecture. Bajpai et al. [18] proposed three models, VGGNet16, RenNet101, and a modified AlexNet, achieving a training accuracy of 99.97%. Olushola Olawuyi and Serestina Viriri [19] proposed a model using ResNet50 for plant disease detection and classification, attaining an accuracy of 98%. Trishita et al. [20] presented a technique incorporating data augmentation, data partitioning, and shuffling along with

deep learning models like SVM, VGG16, and a CNN, achieving accuracies of 87%, 92%, and 98%, respectively. Alok et al. [21] proposed a model that combines a hierarchical deep learning convolutional neural network with fuzzy local binary patterns for feature extraction. Their model achieved an accuracy of 95.77%.

Most of the research has been conducted using the PlantVillage dataset, which was developed for specific geographic regions and climatic conditions. Potato plant diseases can vary globally due to differences in shape, variety, and environmental factors. Additionally, the PlantVillage dataset has a limited number of samples and exhibits an imbalanced distribution across classes. This limitation can impact the generalizability and robustness of disease detection models trained exclusively on these data. Rashid et al. [22] developed a dataset specifically for detecting potato diseases in the South Asian subcontinent, with data collected from the central Punjab region of Pakistan. The dataset includes videos and images captured using various devices such as phone cameras, digital cameras, and drones. Following image collection, pathologists labeled the images into Early Blight, Late Blight, and Healthy leaf categories.

These studies have shown that Convolutional Neural Networks in deep learning models are effective for classifying infected images. It has been proven that transfer learning with pre-trained models improves the model performance. The proposed model incorporates an attention mechanism for images to further enhance the model's performance.

1.2. Motivation

- In CNN-based image classification tasks, feature extraction is achieved using filters or kernels which extract edges, shapes, and textures. These are used to produce more prominent features that aid in classifying different images. Since an image may contain multiple features, not all of which are relevant to classification tasks, a mechanism is needed to focus only on the relevant features. This is where the attention mechanism comes in; it prioritizes specific parts of an image that are crucial for distinguishing between classes while downplaying irrelevant areas. By assigning greater weight to distinguishing features, the attention mechanism enhances the model's ability to classify images accurately.
- In order to capture minute features in the image, a series of convolution layers are used because initial layers only capture prominent features and as we go deeper into the layers, the model is able to capture very specific features as well. A dense network has large number of trainable parameters, which increases the training time. Additionally, the size of the model increases, making it unsuitable for integration in devices with less computation power and memory.
- In a multi-layer CNN model, the large number of trainable parameters makes the model highly complex. This complexity increases the risk of overfitting, making the model difficult to train. Additionally, training a complex model is time consuming due to the need to compute optimal values for numerous parameters.

1.3. Contribution

- We propose an attention-based CNN model that selectively focuses on specific parts of an image to differentiate between healthy and infected. The model utilizes the Convolutional Block Attention Module (CBAM) developed by Sanghyun et al. [23].
- The proposed model emphasizes distinguishing features while minimizing the importance of other features. This helps the model to achieve comparable results without relying on complex architectures, resulting in a simpler model with fewer parameters and lighter weights.
- By focusing on the most informative features of an image, the model significantly reduces the number of parameters. This results in fewer trainable parameters which avoids model overfitting and significantly reduces the training time. Additionally, the simplified model lowers computational costs and eliminates the need for high-end GPUs.

The paper is structured as follows: Section 1 provides the Introduction and Literature Review; Section 2 outlines the Proposed Technique; Section 3 presents the Experimental Results and Analysis, followed by the Conclusion and Future Scope, which are discussed in Section 4.

2. Convolutional Block Attention Module-Based Technique

The proposed technique, illustrated in Figure 1, is a combination of three components: a pre-trained model, attention blocks, and dense layers. We utilize transfer learning to simplify model complexity and reduce the number of trainable parameters. The pre-trained model is imported without its dense layers and all layers are frozen, making them non-trainable. The output from the pre-trained model is fed into the CBAM module, which then passes the data to trainable dense layers for prediction. All the images were by default in the shape of 256 × 256. The resize operation was performed to form a base in the dataset so that if any sample had a different size, it got changed to 256 × 256. Every pixel value was standardized in the range 0 to 1. The standardization method was performed using the Image Data Generator method of Keras library in Python 3.10. The purpose of standardization was to give consistent input; it allows the model to generalize faster when input is consistent. The pixel value of all the images was standardized in the range 0 to 1. The dataset was split into a training, testing, and validation set in the ratio of 80%, 10%, and 10%, respectively. The optimizer used is Adam [24] and categorical cross-entropy is the loss function. The metric used to evaluate the model is accuracy score. No data augmentation techniques were applied; all models were trained on original images. All the models were trained for only 10 epochs as the baseline for comparison.

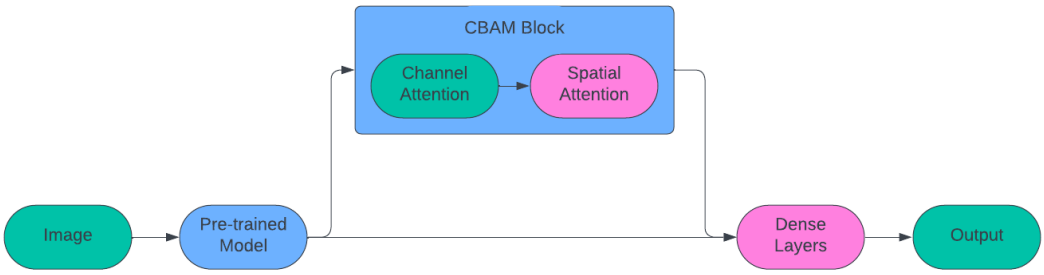


Figure 1. Flowchart of proposed model.

The Convolutional Block Attention Module (CBAM) is a specialized attention mechanism used in Convolutional Neural Networks. It is a combination of two sub-modules: the channel attention module (CAM) and spatial attention module (SAM).

- Channel attention focuses on highlighting relevant and important features across channels. It processes an image or a feature map, which consists of multiple channels, each representing a different pattern or aspect detected by a filter. The feature map is passed through a global average pooling layer, generating an average value for each channel to capture the general importance of each channel across the feature map. Simultaneously, the feature map is passed through a global max pooling layer to identify the max value in each channel. The purpose of the global max pooling layer is to highlight the most prominent features in each channel. The outputs from the global average and global max layers are then fed into dense layers, with the final dense layers containing a number of neurons equal to the number of channels in the feature map. These dense layers assign weights to each channel based on importance, which helps in generating the final attention map. After processing through the dense layers, the outputs are combined using element-wise addition to produce a single feature map with multiple channels, each channel containing a single number representing its importance. The feature map is then passed through a sigmoid activation function to normalize its values. Finally, the normalized feature map is combined with the original feature map through element-wise multiplication, resulting in a feature map

that retains the original feature map with prominent channels having highlighted features. Figure 2 shows the flow of the channel attention mechanism.

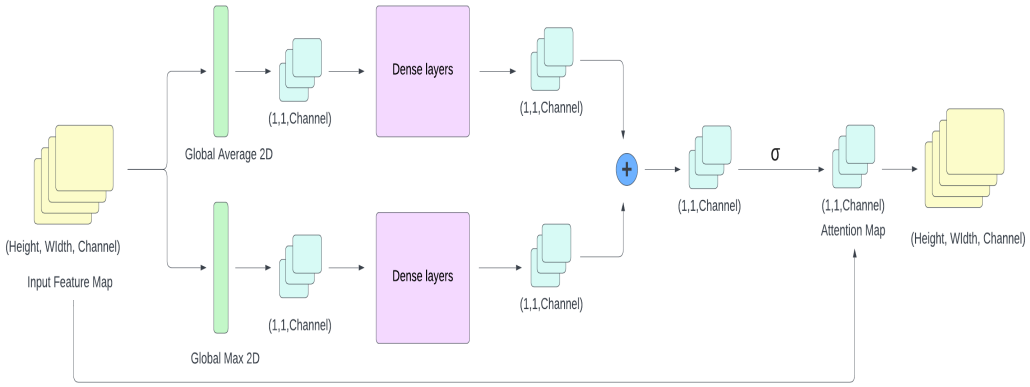


Figure 2. Flow of channel attention module.

- Spatial attention focuses on finding the most dominant regions in an image by identifying the most significant group of pixels. It processes an image or a feature map composed of multiple channels, each channel representing a grid. The feature map is passed through two layers: a global average pooling layer and global max pooling layer. Unlike channel attention, where the average value within each channel is obtained, here the average pooling is performed across the channel axis calculating the average value across all channels for each pixel. Similarly, average max pooling is performed across the channel axis to obtain the maximum value across all channels for each pixel. This results in two feature maps, each highlighting the brightest spots across all channels. These feature maps are then combined to form a two-channel feature map which is then passed through one or more convolutional layers, resulting in a single-channel feature map. This is the attention map highlighting the location of the most important features. This attention feature map is then combined with the original feature map through the element-wise multiplication operation. Shape matching of both feature maps is handled by the broadcasting feature of the Numpy library. Figure 3 illustrates the flow of the channel attention mechanism.

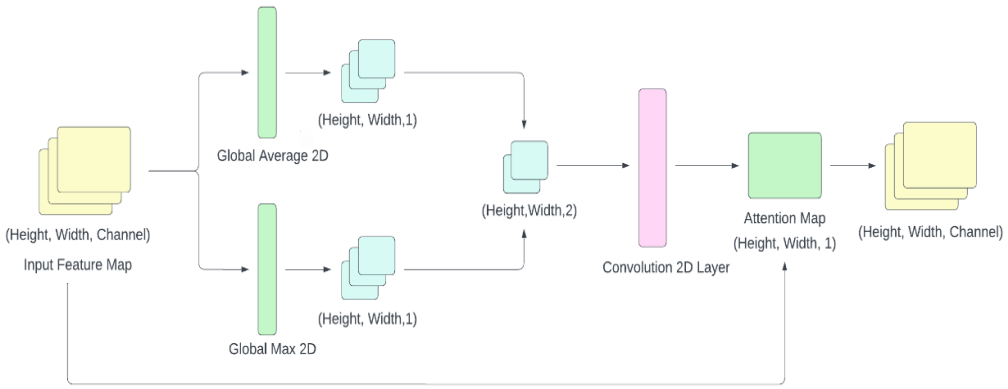


Figure 3. Flow of spatial attention module.

Both the channel attention module and spatial attention module can be implemented either in a parallel or sequential fashion. In our proposed technique, we used a sequential arrangement where channel attention is performed first and its output is then fed into the spatial attention module. In simple terms, the channel attention focuses on finding "what" are the important features in an image and the spatial attention focuses on finding "where" those important features are located within the image.

3. Experimental Results and Analysis

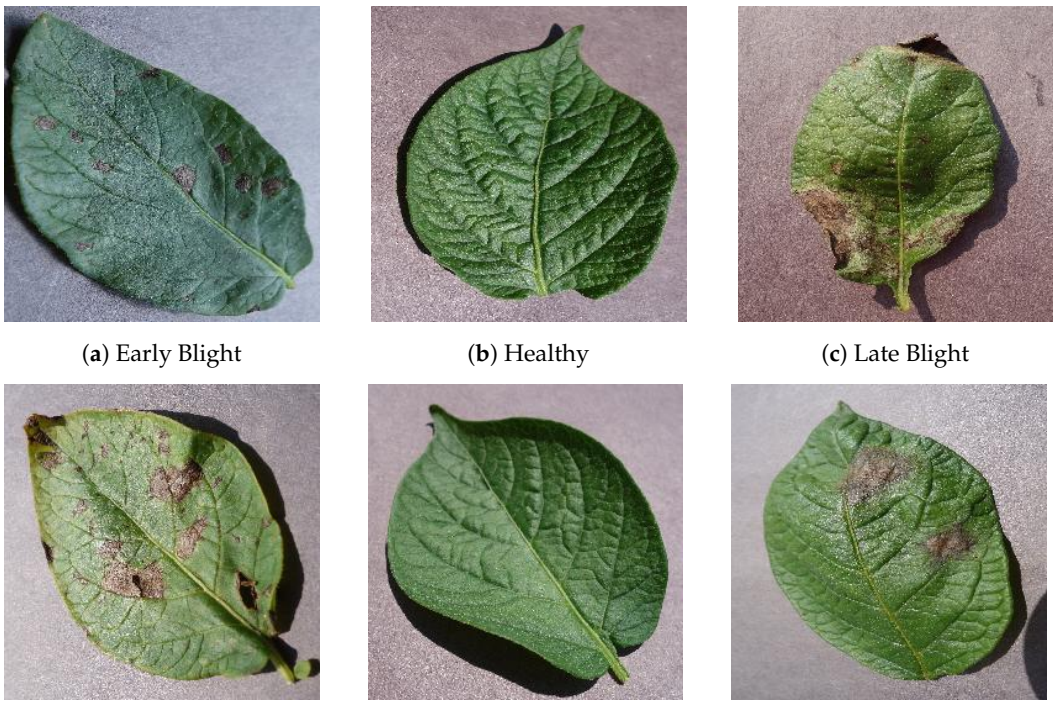
To demonstrate our findings, we combined the PlantVillage and Potato Leaf Dataset (PLD). The proposed technique is implemented using four pre-trained models, DenseNet169 [25], MobileNet [26], XceptionNet [27], and VGG16 [28], on the developed dataset. All the models were trained on a virtual environment, named 'Kaggle'. The CPU used was Intel(R) Xeon(R) CPU @ 2.00 GHz of Skylake generation with two cores (Intel, Santa Clara, CA, USA). The GPU used for training was an NVIDIA Tesla T4 GPU with 16 GB VRAM memory and 2560 cuda cores (NVIDIA, Santa Clara, CA, USA). The ram used was 32 GB. Table 1 shows the summary of parameters which are common for all the presented models.

Table 1. Summary of the parameters used in the models.

Parameter	Value
Batch Size	32
Input Shape	(256, 256)
Optimizer	Adam
Loss Function	Categorical Cross Entropy
Epochs	10

3.1. Dataset Used

In potato disease detection tasks, most research has primarily focused on two datasets: PlantVillage [2] and the Potato Leaf Dataset (PLD) [22]. The PlantVillage dataset comprises a total of 2152 images, with 1000 images each for Early Blight and Late Blight, and 152 images for Healthy plants. Some of the images of this dataset are shown in Figure 4. Table 2 shows the distribution of the PlantVillage dataset, highlighting that the number of samples in the Healthy class is very low as compared to the other classes.



(**a**) Early Blight (**b**) Healthy (**c**) Late Blight

(**d**) Early Blight (**e**) Healthy (**f**) Late Blight

Figure 4. Some of the sample images from the PlantVillage dataset.

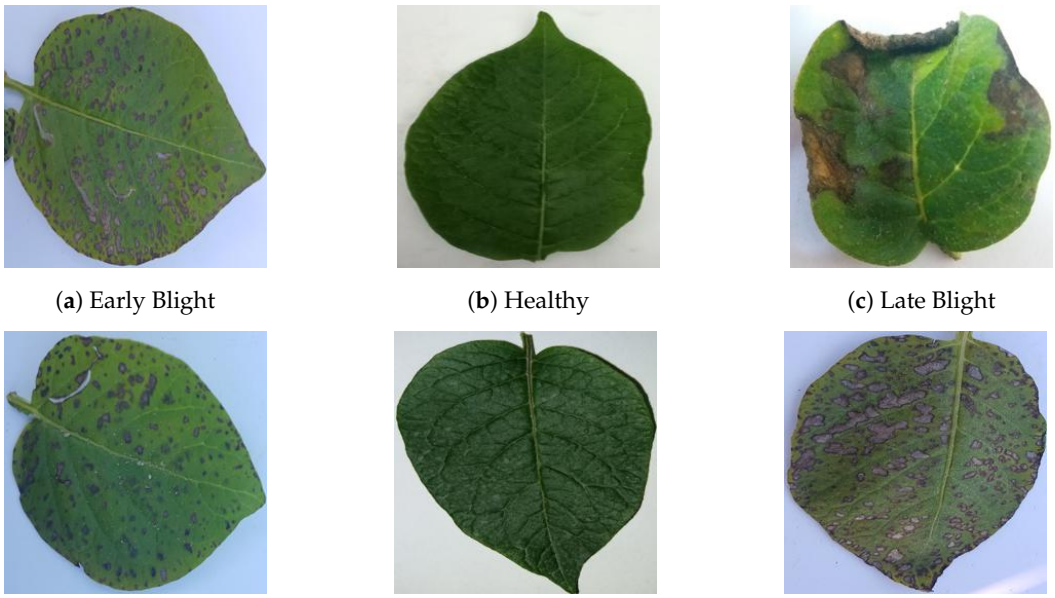
Table 2. Summary of the PlantVillage dataset.

Class Labels	Samples
Early Blight	1000
Late Blight	1000
Healthy	152
Total Samples	2152

The PLD dataset comprises 4062 sample images, with 1628 for Early Blight, 1414 for Late Blight, and 1020 for Healthy plants. Some of the images of this dataset are shown in Figure 5. Table 3 shows the distribution of the PLD dataset. This dataset is imbalanced, as the number of samples in each class varies significantly.

Table 3. Summary of the Potato Leaf Dataset dataset.

Class Labels	Samples
Early Blight	1628
Late Blight	1414
Healthy	1020
Total Samples	4062



(**a**) Early Blight (**b**) Healthy (**c**) Late Blight

(**d**) Early Blight (**e**) Healthy (**f**) Late Blight

Figure 5. Some of the sample images from the PLD dataset.

Finally, both the datasets are merged to create a single, larger dataset, aimed at generalizing disease detection in infected plants. Due to the limited number of samples in the Healthy class, random images from this class are selected, and for each image, an additional image is generated by applying a data augmentation technique [29]. The parameters used for generating a new image are a rotation range of 40 degrees, width and height shift ranges of 0.2, a shear range of 0.2, and a zoom range of 0.2. Additionally, horizontal flipping was also applied, and the fill mode was set to 'nearest'. This process was applied to 887 samples from the Healthy class, increasing the total number of Healthy samples to 2059. The number of samples for Early Blight and Late Blight are 2628 and 2414, respectively. The total number of images in the combined dataset is 7101. Table 4 shows the distribution of the combined dataset.

Table 4. Summary of the combined PlantVillage and PLD dataset.

Class Labels	Samples
Early Blight	2628
Late Blight	2414
Healthy	2059
Total Samples	7101

3.2. DenseNet169

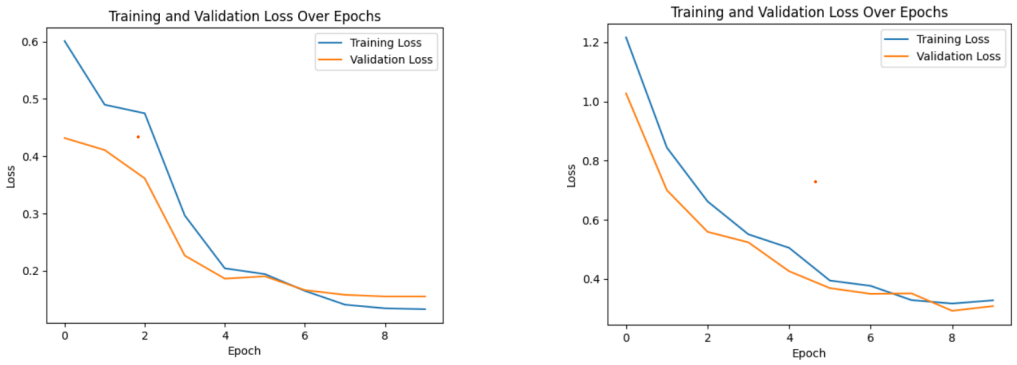
The attention-based DenseNet169 architecture includes two dense layers. The first dense layer has 32 units with ReLU activation and an L_2 regularizer with a factor of 0.01. The second dense layer has three units with softmax activation while the traditional DenseNet169 architecture includes four dense layers. The first dense layer has 512 units with ReLU activation and an L_2 regularizer with a factor of 0.001. The second dense layer has 256 units with ReLU activation and an L_2 regularizer with a factor of 0.001. The third dense layer has 128 units with ReLU activation and an L_2 regularizer with a factor of 0.001. The final dense layer has three units with softmax activation. It uses 1,022,211 parameters to achieve an accuracy of 97.47%. In comparison, the attention-based DenseNet169 model, which uses 611,147 parameters, achieves a similar accuracy of 97.17%. The attention-based model is able to reduce the number of parameters by 40%.

Table 5 shows the accuracy distribution for the DenseNet169 model both with and without the attention module.

Table 5. Attention vs. without attention comparison of accuracy in DenseNet169 architecture.

	With Attention			Without Attention		
	Training	**Validation**	**Testing**	**Training**	**Validation**	**Testing**
Accuracy	97.47	96.83	96.12	97.17	97.78	97.58

Figure 6 illustrates the loss reduction for the DenseNet169 module with and without the attention approach. Figure 6a shows the loss reduction when the DenseNet169 module is combined with the attention module while Figure 6b depicts the loss reduction when the DenseNet169 model is implemented without any form of attention. It is evident that after 10 epochs the decrease in loss is much lower in the case of the attention-based technique. By employing the attention approach, we successfully reduced the loss to below 0.2, whereas the model without attention remains around a loss value of 0.4. Table 6 shows the loss values after training for 10 epochs.



(**a**) With attention loss curve (**b**) Without attention loss curve

Figure 6. DenseNet169 loss curve for attention and without attention module.

Table 6. Attention vs. without attention comparison of loss in DenseNet169 architecture.

	With Attention		Without Attention	
	Training	**Validation**	**Training**	**Validation**
Loss	0.1329	0.1551	0.3273	0.3078

The results can be further extended to other metrics such as precision, recall, and F_1 score. The attention-based DenseNet169 model achieved a precision score of around 95.24%, while the without-attention DenseNet169 model achieved 96.44%. For recall, the attention-based DenseNet169 module scored 95.93%, while the model without attention achieved 96.55%. The attention-based DenseNet169 model achieved an F_1 score of 95.56%, compared to 96.49% for the model without attention. The precision, recall, and F_1 score are calculated by the averaging the values for each individual class. Table 7 shows the distribution of these metrics.

Table 7. Summary table for DenseNet169.

	With Attention	**Without Attention**
Precision	95.24	96.44
Recall	95.93	96.55
F_1 Score	95.56	96.49

3.3. MobileNet

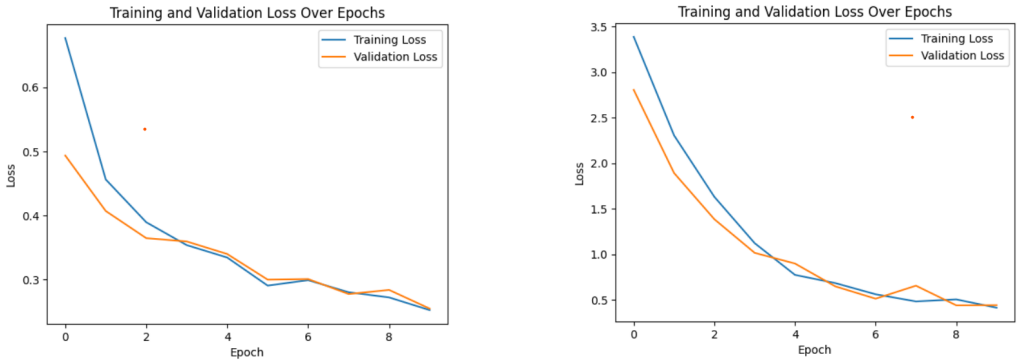
The attention-based MobileNet architecture consists of three dense layers: the first with 128 units and ReLU activation, and an L_2 regularizer of 0.001; the second with 64 units, also using ReLU activation and the same regularizer; and the final layer with 3 units and softmax activation. Meanwhile, the traditional MobileNet architecture consists of four dense layers: the first three layers each have 512 units with ReLU activation and an L_2 regularizer of 0.002, while the fourth layer has 3 units with softmax activation. The MobileNet model uses 1,056,771 trainable parameters to achieve an accuracy of 95.96%. In contrast, the attention-based MobileNet model with 405,477 trainable parameters achieves an accuracy of 96.09%. This attention-based model reduces the number of trainable parameters by approximately 62%.

Table 8 shows the accuracy distribution for the MobileNet model implemented with and without attention.

Table 8. Attention vs. without attention comparison of accuracy in MobileNet architecture.

	With Attention			Without Attention		
	Training	**Validation**	**Testing**	**Training**	**Validation**	**Testing**
Accuracy	96.09	95.25	97.25	95.96	94.14	94.67

Figure 7 shows the decrease in the loss when implementing the MobileNet module with and without attention. Figure 7a shows the decrease in loss when the attention module combined with MobileNet is implemented and Figure 7b shows the decrease in loss when the MobileNet model is implemented without any form of attention. Clearly, we can observe that after 10 epochs, the decrease in loss is much lower in case of the attention-based technique; using attention, we were able to reduce loss below the level of 0.3, while the model without attention was still struggling around the value of 0.5. Table 9 shows the loss values after training for 10 epochs.



(**a**) With attention loss curve (**b**) Without attention loss curve

Figure 7. MobileNet loss curve for attention and without attention module.

Table 9. Attention vs. without attention comparison of loss in MobileNet architecture.

	With Attention		**Without Attention**	
	Training	**Validation**	**Training**	**Validation**
Loss	0.2526	0.2549	0.4147	0.4429

The results can be further extended to other metrics also like precision, recall, and F_1 score. The attention-based MobileNet model achieved a precision score of around 96.84% while the without-attention MobileNet model achieved 94.82%. The attention module combined with MobileNet was able to achieve a recall score of 97.15% while the without-attention model achieved 92.90%. The attention-based MobileNet model achieved an F_1 score of 96.99%, while the without-attention model achieved 93.70%. The precision, recall, and F_1 score are calculated by taking the average value of each individual class. Table 10 shows the distribution of the mentioned metrics.

Table 10. Summary table for MobileNet.

	With Attention	**Without Attention**
Precision	96.84	94.82
Recall	97.15	92.90
F_1 Score	96.99	93.70

3.4. XceptionNet

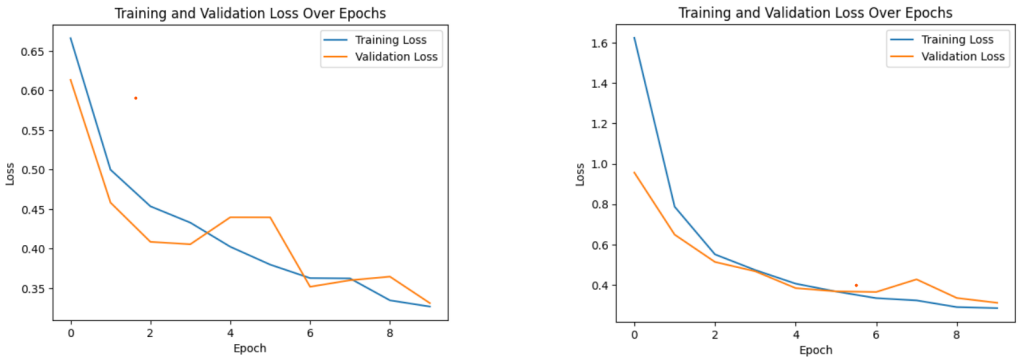
The attention-based XceptionNet architecture includes three dense layers: the first and second layers each have 64 units with ReLU activation and an L_2 regularizer of 0.001. The final layer has 3 units with softmax activation while the traditional XceptionNet architecture consists of six dense layers: the first five layers each have 512 units with ReLU activation and an L_2 regularizer of 0.001, while the sixth layer has 3 units with softmax activation. The XceptionNet model uses 2,101,251 parameters to attain an accuracy of 93.62%. A similar accuracy of 92.70% is achieved by the attention-based XceptionNet model which uses 1,190,821 trainable parameters. The attention-based approach reduces the number of parameters by approximately 44%.

Table 11 shows the accuracy distribution for the MobileNet model with and without the attention approach.

Table 11. Attention vs. without attention comparison of accuracy in XceptionNet architecture.

	With Attention			Without Attention		
	Training	**Validation**	**Testing**	**Training**	**Validation**	**Testing**
Accuracy	92.70	93.19	93.70	93.62	92.87	93.70

Figure 8 illustrates the decrease in loss for the XceptionNet with and without the attention approach. Figure 8a shows the loss reduction when the attention approach is combined with XceptionNet, while Figure 8b depicts the loss decrease for the XceptionNet model without any form of attention. It is evident that after 10 epochs, the attention-based technique results in a more substantial loss reduction, bringing it below 0.35, whereas the model without attention remains around a loss value of 0.4. Table 12 depicts the loss values after 10 epochs of training.



(**a**) With attention loss curve (**b**) Without attention loss curve

Figure 8. XceptionNet loss curve for attention and without attention module.

Table 12. Attention vs. without attention comparison of loss in XceptionNet architecture.

	With Attention		Without Attention	
	Training	**Validation**	**Training**	**Validation**
Loss	0.3267	0.3310	0.2852	0.3116

The results can be further evaluated using other metrics such as precision, recall, and F_1 score. The attention-based XceptionNet model achieved a precision score of approximately 92.92% compared to 92.88% for the model without attention. For recall, the attention-based module scored 93.54%, while the model without attention achieved 93.07%. The attention-based XceptionNet model attained an F_1 score of 93.21%, whereas the model without attention achieved 92.96%. These metrics are calculated by averaging the values for each individual class. Table 13 shows the distribution of these metrics.

Table 13. Summary table for XceptionNet.

	With Attention	Without Attention
Precision	92.92	92.88
Recall	93.54	93.07
F_1 Score	93.21	92.96

3.5. VGG16

The attention-based VGG16 architecture includes three dense layers: the first layer has 128 units with ReLU activation and no L_2 regularization, the second layer has 64 units with

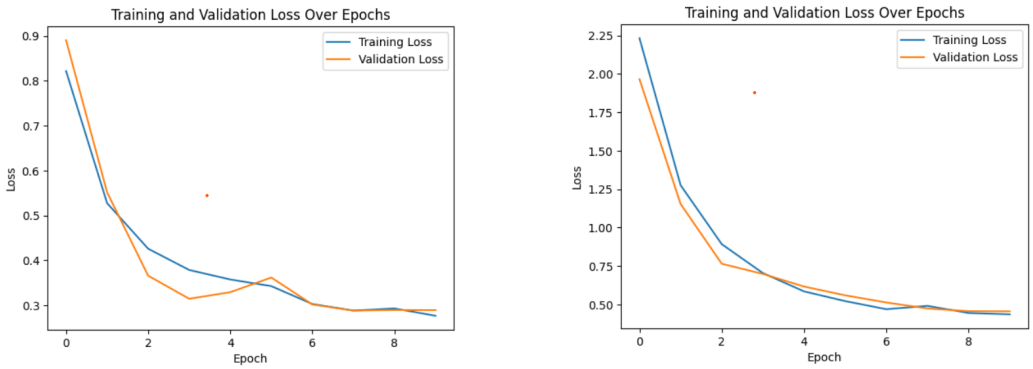
ReLU activation and no L_2 regularization, and the third layer has 3 units with softmax activation; meanwhile, the traditional VGG16 architecture consists of two dense layers: the first layer has 512 units with ReLU activation and an L_2 regularizer of 0.005, while the second layer has 3 units with softmax activation. The VGG16 model utilizes 266,243 parameters to achieve an accuracy of 90.46%. In comparison, attention-based VGG16 model achieves almost similar accuracy of 89.22% with only 141,733 parameters, reducing the number of parameters by approximately 47%.

Table 14 shows the accuracy distribution of the VGG16 model implemented with and without an attention module.

Table 14. Attention vs. without attention comparison of accuracy in VGG16 architecture.

	With Attention			**Without Attention**		
	Training	**Validation**	**Testing**	**Training**	**Validation**	**Testing**
Accuracy	89.22	88.91	90.32	90.46	89.86	90.48

Figure 9 illustrates the decrease in loss for the VGG16 module with and without the attention module. Figure 9a shows the loss reduction when the attention module is combined with VGG16 and Figure 9b shows the loss decrease for the VGG16 model without any form of attention. Clearly, one can observe that after 10 epochs, the attention-based technique results in a more substantial loss reduction, bringing it below 0.3, whereas the model without attention remains around a loss value of 0.5. Table 15 shows the loss values after training for 10 epochs.



(**a**) With attention loss curve (**b**) Without attention loss curve

Figure 9. VGG16 loss curve for attention and without attention module.

Table 15. Attention vs. without attention comparison of loss in VGG16 architecture.

	With Attention		**Without Attention**	
	Training	**Validation**	**Training**	**Validation**
Loss	0.2813	0.2872	0.4361	0.4556

The results can be extended to other metrics such as precision, recall, and F_1 score. The attention-based VGG16 model achieved a precision score of approximately 89.62%, compared to 90.03% for the model without attention. For recall, the attention-based VGG16 model scored 89.36%, while the model without attention achieved 89.68%. The attention-based VGG16 model attained an F_1 score of 89.42%, compared to 89.71% for the model without attention. These metrics are calculated by averaging the values for each individual class. Table 16 shows the distribution of these metrics.

Table 16. Summary table for VGG16.

	With Attention	Without Attention
Precision	89.62	90.03
Recall	89.36	89.68
F_1 Score	89.42	89.71

3.6. Comparison of the Inference Time

The average inference time for each model with CBAM attention is calculated and compared with corresponding models without attention. There were 620 samples in the test data and the prediction time is calculated for each sample and its average is taken. The authors have observed a slight increase in inference time, which is rational because the CBAM introduces additional operations, such as channel attention and spatial attention. These operations require extra computations, which contribute to increased inference time. Table 17 shows the time taken by each model during inference.

Table 17. Comparison of the inference time with and without the attention module.

	With Attention	Without Attention
DenseNet169	42 ms	32 ms
MobileNet	34 ms	22 ms
XceptionNet	32 ms	21 ms
VGG16	23 ms	18 ms

4. Conclusions and Future Work

The proposed work demonstrates that utilizing an attention-based approach can achieve similar accuracy with reduced model complexity. By embedding the attention concept in the deep learning model, the number of trainable parameters in the MobileNet model can be reduced by approximtaly 62%. Additionally, this approach maintains comparable precision, recall, and F_1 score values with fewer parameters. The proposed technique can be extended to build state-of-the-art models with improved performance and fewer trainable parameters. The simplified models with fewer parameters enables the development of light weight, portable applications for devices with low computational power. For example, mobile applications can be developed to allow farmers to take a picture of a leaf and detect whether the plant is healthy or infected. This would be particularly useful for farmers who lack technical knowledge and cannot afford high-end computational devices. Similarly, light weight websites could be deployed to allow users to upload images of a leaves and distinguish between healthy and infected crops. The technique can be further enhanced by incorporating various other technologies such as quantum computation, deep learning architectures, vision–language models, etc.

Author Contributions: Methodology, A.S.R.; Formal analysis, A.S.R.; Investigation, S.S.K. and A.S.R.; Writing—original draft, A.S.R.; Writing—review & editing, S.S.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The data presented in this study are openly available in reference number [2,22].

Conflicts of Interest: The authors declare no conflict of interest.

References

1. McNicoll, G. The United Nations' Long-Range Population Projections. *Popul. Dev. Rev.* **1992**, *18*, 333. [CrossRef]
2. Hughes, D.; Salathé, M. An open access repository of images on plant health to enable the development of mobile disease diagnostics. *arXiv* **2015**, arXiv:1511.08060.
3. Baker, N.; Capel, P. *Environmental Factors that Influence the Location of Crop Agriculture in the Conterminous United States*; US Department of the Interior, US Geological Survey: Reston, VA, USA, 2011.
4. Khan, A.; Vibhute, A.D.; Mali, S.; Patil, C.H. A systematic review on hyperspectral imaging technology with a machine and deep learning methodology for agricultural applications. *Ecol. Inform.* **2022**, *69*, 101678. [CrossRef]
5. Dubey, S.; Jalal, A. Adapted Approach for Fruit Disease Identification Using Images. In *Image Processing: Concepts, Methodologies, Tools, and Applications*; IGI Global: Hershey, PA, USA, 2013; pp. 1395–1409.
6. Li, G.; Ma, Z.; Wang, H. Image Recognition of Grape Downy Mildew and Grape. In Proceedings of the International Conference on Computer and Computing Technologies in Agriculture, Beijing, China, 29–31 October 2011; pp. 151–162.
7. Rauf, H.; Saleem, B.; Lali, M.; Khan, M.; Sharif, M.; Bukhari, S. A citrus fruits and leaves dataset for detection and classification of citrus diseases through machine learning. *Data Brief* **2019**, *26*, 104340. [CrossRef] [PubMed]
8. Sujatha, R.; Chatterjee, J.; Jhanjhi, N.; Brohi, S. Performance of deep learning vs. machine learning in plant leaf disease detection. *Microprocess. Microsyst.* **2021**, *80*, 103615. [CrossRef]
9. Geetharamani, G.; Pandian, A. Identification of plant leaf diseases using a nine-layer deep convolutional neural network. *Comput. Electr. Eng.* **2019**, *76*, 323–338.
10. Khamparia, A.; Saini, G.; Gupta, D.; Khanna, A.; Tiwari, S.; de Albuquerque, V. Seasonal Crops Disease Prediction and Classification Using Deep Convolutional Encoder Network. *Circuits Syst. Signal Process.* **2019**, *39*, 818–836. [CrossRef]
11. Liang, Q.; Xiang, S.; Hu, Y.; Coppola, G.; Zhang, D.; Sun, W. PD2SE-Net: Computer-assisted plant disease diagnosis and severity estimation network. *Comput. Electron. Agric.* **2019**, *157*, 518–529. [CrossRef]
12. Khalifa, N.; Taha, M.; Abou El-Maged, L.; Hassanien, A. Artificial Intelligence in Potato Leaf Disease Classification: A Deep Learning Approach. In *Machine Learning and Big Data Analytics Paradigms: Analysis, Applications and Challenges*; Springer: Berlin/Heidelberg, Germany, 2021; pp. 63–79.
13. Rozaqi, A.; Sunyoto, A. Identification of Disease in Potato Leaves Using Convolutional Neural Network (CNN) Algorithm. In Proceedings of the 2020 3rd International Conference on Information and Communications Technology (ICOIACT), Yogyakarta, Indonesia, 24–25 November 2020; pp. 72–76.
14. Iqbal, M.; Talukder, K. Detection of potato disease using image segmentation and machine learning. In Proceedings of the 2020 International Conference on Wireless Communications Signal Processing and Networking (WiSPNET), Chennai, India, 4–6 August 2020; pp. 43–47.
15. Singh, A.; Kaur, H. Potato plant leaves disease detection and classification using machine learning methodologies. *IOP Conf. Ser. Mater. Sci. Eng.* **2021**, *1022*, 012121. [CrossRef]
16. Chakraborty, K.; Mukherjee, R.; Chakroborty, C.; Bora, K. Automated recognition of optical image based potato leaf blight diseases using deep learning. *Physiol. Mol. Plant Pathol.* **2022**, *117*, 101781. [CrossRef]
17. Mahum, R.; Munir, H.; Mughal, Z.; Awais, M.; Sher Khan, F.; Saqlain, M.; Mahamad, S.; Tlili, I. A novel framework for potato leaf disease detection using an efficient deep learning model. *Hum. Ecol. Risk Assess. Int. J.* **2023**, *29*, 303–326. [CrossRef]
18. Bajpai, A.; Tyagi, M.; Khare, M.; Singh, A. A Robust and Accurate Potato Leaf Disease Detection System Using Modified AlexNet Model. In Proceedings of the 2023 9th International Conference on Computer and Communication Engineering (ICCCE), Kuala Lumpur, Malaysia, 15–16 August 2023; pp. 1–5. [CrossRef]
19. Olawuyi, O.; Viriri, S. Plant Diseases Detection and Classification Using Deep Transfer Learning. In Proceedings of the Pan-African Artificial Intelligence and Smart Systems Conference, Dakar, Senegal, 2–4 November 2022; pp. 270–288.
20. Acharjee, T.; Das, S.; Majumder, S. Potato Leaf Diseases Detection Using Deep Learning. *Int. J. Digit. Technol.* **2023**, *11*, 3.
21. Kumar, A.; Patel, V. Classification and identification of disease in potato leaf using hierarchical based deep learning convolutional neural network. *Multimed. Tools Appl.* **2023**, *82*, 31101–31127. [CrossRef]
22. Rashid, J.; Khan, I.; Ali, G.; Almotiri, S.H.; AlGhamdi, M.A.; Masood, K. Multi-Level Deep Learning Model for Potato Leaf Disease Recognition. *Electronics* **2021**, *10*, 2064. [CrossRef]
23. Woo, S.; Park, J.; Lee, J.; Kweon, I. CBAM: Convolutional Block Attention Module. In *Proceedings of the Computer Vision—ECCV 2018*; Ferrari, V., Hebert, M., Sminchisescu, C., Weiss, Y., Eds.; Springer: Cham, Switzerland, 2018; pp. 3–19. [CrossRef]
24. Kingma, D.P.; Ba, J.L. Adam: A Method for Stochastic Optimization. In Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015), San Diego, CA, USA, 7–9 May 2015.
25. Huang, G.; Liu, Z.; van der Maaten, L.; Weinberger, K.Q. Densely Connected Convolutional Networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 21–26 July 2017.
26. Howard, A.G.; Zhu, M.; Chen, B.; Kalenichenko, D.; Wang, W.; Weyand, T.; Andreetto, M.; Adam, H. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv* **2017**, arXiv:1704.04861.
27. Chollet, F. Xception: Deep Learning with Depthwise Separable Convolutions. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 21–26 July 2017; pp. 1251–1258.

28. Simonyan, K.; Zisserman, A. Very Deep Convolutional Networks for Large-Scale Image Recognition. In Proceedings of the International Conference on Learning Representations (ICLR), San Diego, CA, USA, 7–9 May 2015.
29. Krizhevsky, A.; Sutskever, I.; Hinton, G.E. ImageNet Classification with Deep Convolutional Neural Networks. *Adv. Neural Inf. Process. Syst.* **2012**, *25*, 1097–1105. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.